

NT230

CƠ CHẾ HOẠT ĐỘNG CỦA MÃ ĐỘC

GV: TS. Phan Thế Duy

Trường Đại học Công nghệ Thông tin
Đại học Quốc gia Thành phố Hồ Chí Minh

BUỔI 03 · LÂY FILE VÀ THỰC THI TRONG TIẾN TRÌNH





Buổi 03

Lây file và thực thi trong tiến trình

Câu hỏi xuyên suốt

Một mẫu có thể thay đổi tệp hoặc chiếm quyền thực thi trong tiến trình bằng những cơ chế nào? Bằng chứng nào giúp ta phân biệt hai trường hợp?

- 90 phút trên lớp; dùng mẫu minh họa và môi trường cô lập.
- Đầu ra: sơ đồ chuỗi hành vi, vị trí bằng chứng và kết luận có giới hạn.

Nội dung chính

- 1 Bối cảnh và mục tiêu
- 2 Virus lây file
- 3 DLL loading và injection
- 4 Process hollowing
- 5 Đối chiếu bằng chứng
- 6 Tình huống thực hành và exit ticket

1

Bối cảnh và mục tiêu

Câu hỏi trọng tâm

Khi một tệp PE thay đổi, thay đổi đó nằm ở file, ở memory hay ở cả hai?

Mục tiêu học tập

- Phân biệt virus file, injection, DLL loading và process hollowing theo nơi mã được đưa vào.
- Mô tả chuỗi chọn đích, tạo vùng nhớ, ghi mã, kích hoạt và dấu vết.
- Đối chiếu bằng chứng tĩnh, động và bộ nhớ để tránh kết luận từ một API đơn lẻ.

Tiêu chí thành công

Giải thích được một mẫu bằng sơ đồ có địa chỉ, trạng thái tiến trình và ít nhất hai nguồn bằng chứng.

Gợi mở: cùng một mục tiêu, hai con đường

Cá nhân rồi trao đổi cặp · 4 phút

Một chương trình lành mở ra nhưng sau đó chạy thêm mã không nằm trong file ban đầu. Đề xuất hai giả thuyết: file đã bị sửa trước khi chạy; hoặc mã được đưa vào memory khi chương trình đang chạy.

Sản phẩm: mỗi giả thuyết kèm một bằng chứng có thể kiểm tra.

Gợi ý suy luận

Hỏi “mã ở đâu trước khi chạy?” và “ai tạo ra vùng nhớ hoặc thread mới?”.

Bốn khái niệm cần tách

Virus file

Gắn hoặc chèn mã vào tệp để lần chạy sau mã đó được gọi.

Injection

Đưa dữ liệu hoặc mã vào không gian địa chỉ của process khác.

DLL loading

Làm cho process nạp một module DLL, hợp lệ hoặc đáng ngờ.

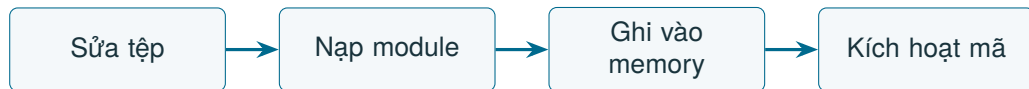
Hollowing

Tạo process hợp lệ rồi thay nội dung image bằng image khác trước khi chạy.

Điểm phân biệt

“Có DLL” hay “có thread từ xa” chỉ là chỉ báo. Cần chứng minh quan hệ nhân quả bằng timeline và trạng thái memory.

Bản đồ bài học



- Phần 2: virus file và các kiểu chèn mã vào tệp.
- Phần 3: DLL injection và process hollowing.
- Phần 4: điều tra bằng chứng và tình huống tổng hợp.

2

Virus lây file

Câu hỏi trọng tâm

Một file bị thay đổi như thế nào để lần chạy sau gọi được mã lạ?

Định nghĩa và điều kiện của virus file

Định nghĩa

Virus file là mã độc tự sao chép hoặc được sao chép vào tệp chủ, sau đó điều khiển luồng thực thi để mã của virus chạy khi tệp chủ được mở.

- Cần cơ chế chọn tệp đích và ghi dữ liệu vào tệp.
- Cần điểm chuyển điều khiển sang mã virus, rồi có thể quay lại mã gốc.
- “Tệp bị sửa” chưa đủ để kết luận virus nếu chưa thấy khả năng tự nhân bản hoặc cơ chế thực thi.

Overwrite, append và cavity

Overwrite	Ghi đè một vùng mã hoặc dữ liệu hiện có; đơn giản nhưng dễ làm hỏng chương trình.
Append	Thêm mã ở cuối file, rồi sửa entry point hoặc stub để chuyển điều khiển.
Cavity	Tận dụng vùng trống bên trong section hoặc padding để giảm thay đổi kích thước file.

Câu hỏi phân tích

Kiểu nào dễ tạo sai khác rõ trong kích thước file? Kiểu nào cần kiểm tra quyền và vùng trống của section?

EPO: entry-point obscuring

Khái niệm

EPO làm cho điểm vào quan sát được không trở trực tiếp đến mã virus. Virus có thể móc vào một lệnh hoặc đường gọi được thực thi sớm hơn.

- Entry point vẫn có thể hợp lệ về mặt PE nhưng luồng thực thi đã bị đổi.
- Cần theo dõi nhánh và lời gọi trước khi chương trình trở về logic gốc.
- Không suy ra EPO chỉ từ entry point “lạ”; cần đối chiếu file gốc hoặc mẫu tương đương.

Ví dụ có hướng dẫn: file trước và sau lây

Quan sát tĩnh

AddressOfEntryPoint đổi; section cuối tăng raw size; checksum khác.

Suy luận có điều kiện

Có dấu hiệu sửa cấu trúc và điểm vào. Cần chạy để xem mã mới có thực thi và có quay về entry point cũ không.

Kết luận đúng mức

“Mã có thay đổi phù hợp với cơ chế chèn mã” mạnh hơn “mẫu chắc chắn là virus”.

Kiểm tra nhanh: dấu hiệu hay bằng chứng?

Cá nhân · 3 phút

Một file tăng 8 KB, section cuối có quyền RWX và entry point chuyển vào vùng mới. Chọn: A chắc chắn là virus; B có chuỗi dấu hiệu cần tái dựng; C file chắc chắn bị lỗi.

Đáp án và cách lập luận

Đáp án: B

Ba quan sát liên quan đến việc thêm vùng mã và chuyển điều khiển, nhưng chưa chứng minh khả năng tự lây.

- Kiểm tra bytes tại vùng mới, disassembly và các nhánh đến vùng đó.
- Chạy trong snapshot, ghi process tree, module, thread và hành vi file.
- So sánh bản lành cùng phiên bản, hash và cấu hình build nếu có.

Phản ví dụ

Packer hoặc bộ cài hợp lệ cũng có thể tạo section quyền cao và entry point stub.

Nguyên tắc: bảo toàn host program

Mục tiêu của nhiều virus file

Mã chèn phải giành quyền điều khiển đủ sớm, thực hiện phần việc của nó, sau đó trả quyền điều khiển về luồng gốc để chương trình vẫn hoạt động.



Lưu đồ lây file ở mức khái niệm

- 1 Liệt kê tệp đích theo phần mở rộng, quyền và trạng thái.
- 2 Đọc header và kiểm tra đã lây chưa.
- 3 Chọn vùng chứa hoặc tạo vùng mã mới, cập nhật kích thước.
- 4 Ghi payload, lưu entry point cũ và điều chỉnh metadata.
- 5 Đóng tệp, tính hash và kiểm tra tệp còn chạy được.

Điểm kiểm chứng

Mỗi bước phải có artifact: danh sách đích, diff header, vùng bytes thay đổi, log mở/ghi và kết quả chạy.

Append: vai trò của stub

Stub

Đoạn mã trung gian nhận quyền điều khiển, chuẩn bị môi trường cho payload rồi chuyển tiếp đến entry point gốc.

- Stub phải biết địa chỉ hoặc cách tính địa chỉ của mã gốc.
- Khi đổi section, phải giữ nhất quán raw size, virtual size, alignment và quyền thực thi.
- Sai alignment có thể làm loader ánh xạ khác dự kiến hoặc làm file không chạy.

Cavity: điều kiện để không phá vỡ image

- Vùng trống phải nằm trong section đã được ánh xạ vào memory.
- Vùng đó phải đủ dung lượng và có quyền phù hợp với mã.
- Không được ghi đè relocation, exception data, import hoặc dữ liệu được chương trình sử dụng.
- Kiểm tra cả file offset lẫn RVA sau khi chèn.

Phản ví dụ

Bytes bằng 0 ở cuối file chưa chắc là cavity có thể thực thi, vì chúng có thể không thuộc phần loader map vào memory.

Đổi chiều địa chỉ sau khi chèn

Công thức nhắc lại

$RVA = Section.VirtualAddress + (FileOffset - PointerToRawData);$

$VA = ImageBase + RVA.$

Cập đôi · 5 phút

Section VA = 0x3000, raw pointer = 0x800, file offset mới = 0xC00, ImageBase = 0x140000000.
Tính RVA và VA.

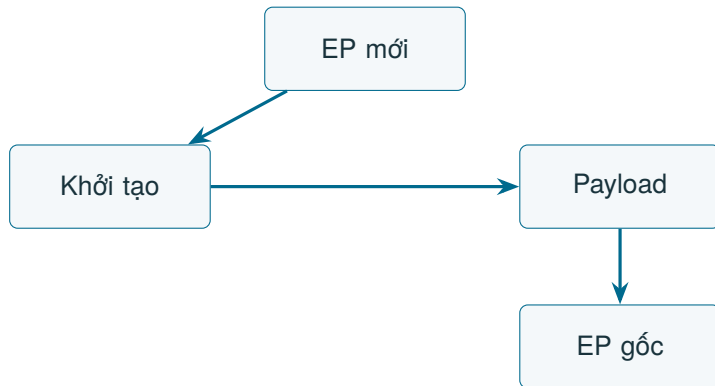
Đáp án tính địa chỉ

- Offset tương đối: $0xC00 - 0x800 = 0x400$.
- RVA: $0x3000 + 0x400 = 0x3400$.
- VA: $0x140000000 + 0x3400 = 0x140003400$.

Ý nghĩa

Nếu disassembly nhảy đến địa chỉ khác, kiểm tra ImageBase thực tế, ASLR và relocation trước khi gọi đó là sai khác.

Tái dựng luồng điều khiển bằng đồ thị



Cách đọc

Đường đi quan sát được quan trọng hơn tên section: xác định node, cạnh, điều kiện nhánh và điểm quay về.

Bài tập phân tích file diff

Nhóm 3 người · 7 phút

Được cung cấp header trước/sau, bảng section và ảnh disassembly. Đánh dấu: metadata thay đổi, bytes mới, điểm chuyển điều khiển và bằng chứng còn thiếu.

Sản phẩm

Bảng theo mẫu “quan sát → ý nghĩa → cách kiểm chứng”.

PE metadata cần đối chiếu

- AddressOfEntryPoint.
- Section raw/virtual size và pointer.
- Characteristics: R, W, X.
- ImageBase, alignment, imports và relocation.

Quyền section và ý nghĩa

R-X	Có thể chứa mã chạy.
RW-	Có thể chứa dữ liệu ghi được.
RWX	Cho phép ghi và thực thi, cần giải thích nhưng không tự kết luận.

Entry point trước và sau

- Lưu giá trị cũ trước khi phân tích.
- Ánh xạ entry point vào section.
- Disassemble vùng mới và lần ngược đến điểm gọi.
- Kiểm tra có đường quay về mã gốc không.

Tính nhất quán raw và virtual

Nguyên tắc

Dữ liệu trên đĩa và vùng ánh xạ trong memory có kích thước, alignment và quyền khác nhau.

- File diff phải được diễn giải cùng section table.
- Bytes ngoài virtual size có thể không được loader map.

Khi nào kết luận là virus?

- Có cơ chế tự tìm/chọn tệp đích.
- Có thao tác sao chép hoặc chèn payload.
- Có chuyển điều khiển khi tệp chạy.
- Có bằng chứng lặp lại trên mẫu hoặc đối chứng.

Phản ví dụ: packer hợp lệ

Tình huống

Packer có thể thay đổi entry point, thêm stub và tạo section có entropy cao để nạp chương trình gốc.

- Vì vậy cần phân biệt “được đóng gói” với “được lây”.
- Bằng chứng tự nhân bản và chọn tệp là điểm phân biệt quan trọng.

Bài tập: file infection hay packer?

Cặp đôi · 5 phút

Một mẫu có stub, section mới và không tạo file khác trong lần chạy. Hãy nêu dữ liệu cần tìm để phân biệt packer với virus file.

Đáp án bài tập

- Tìm thao tác mở và ghi các file khác.
- Theo dõi entry point gốc và payload sau khi unpack.
- Kiểm tra lặp lại trên nhiều file đích.
- So sánh với packer lành cùng cấu hình.

Tóm tắt kỹ thuật phần file

- File infection thay đổi bytes và luồng điều khiển trên đĩa.
- Loader chỉ map phần phù hợp vào memory.
- File offset, RVA, VA và section permissions nối được file với memory.
- Packer là phần ví dụ quan trọng cho kết luận quá mức.

Virus lây file và PE injection

Tiêu chí	Virus lây file	PE injection
Đối tượng tác động	Tập PE trên đĩa	Không gian nhớ của process đích
Thời điểm chính	Trước lần chạy tiếp theo của tập đã sửa	Khi process đích đang chạy hoặc vừa được tạo
Thay đổi bền vững	Thường để lại thay đổi trong file	Có thể chỉ tồn tại trong memory
Bằng chứng ưu tiên	Hash/diff, section, entry point, thao tác ghi file	Process handle, vùng memory, PE header/section, thread/context
Phân biệt cốt lõi	Mã được chèn vào file chủ	Các phần của PE được map/ghi vào process khác

Lưu ý

Cả hai có thể làm mã chạy trong process. Hãy xác định nơi payload được đưa vào trước: file trên đĩa hay address space của process.

3

DLL loading và injection

Câu hỏi trọng tâm

Một module được nạp vào process bằng đường hợp lệ hay bị ép nạp?

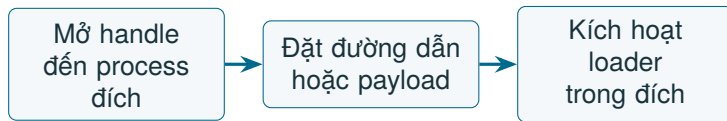
DLL và module trong process

DLL

Dynamic-link library là tệp chứa mã hoặc dữ liệu có thể được nhiều process nạp dùng chung.

- Loader ghi nhận module trong danh sách của process và ánh xạ các section vào memory.
- DLL hợp lệ có thể được nạp khi khởi động hoặc khi chương trình cần một chức năng.
- Phân tích cần kết hợp tên đường dẫn, chữ ký, hash, parent process và thời điểm nạp.

DLL injection: chuỗi cơ chế



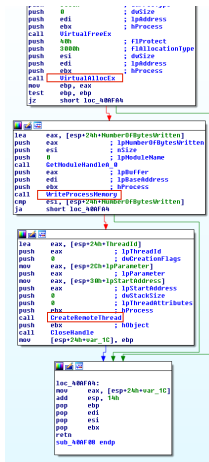
Mẫu kinh điển

OpenProcess → VirtualAllocEx → WriteProcessMemory → CreateRemoteThread/LoadLibrary.

Cảnh báo phân tích

Tên API mô tả khả năng, không tự nó chứng minh ý đồ hay thành công.

DLL injection trong mẫu Rebhip



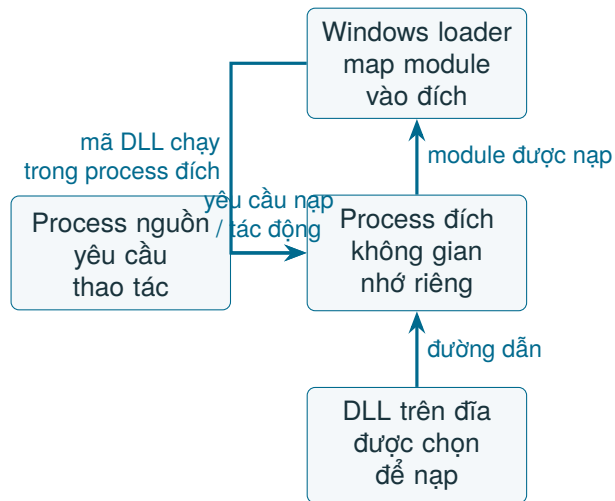
- 1 Cấp phát vùng nhớ trong process đích.
- 2 Ghi đường dẫn DLL vào vùng nhớ đó.
- 3 Tạo remote thread để gọi loader với địa chỉ đường dẫn.

Đọc hình có giới hạn

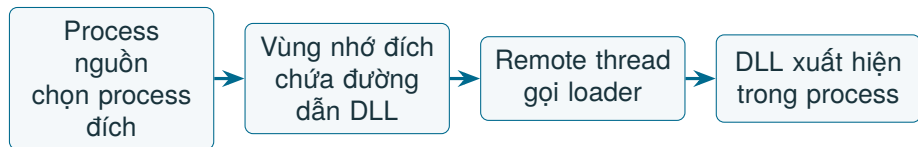
Disassembly minh họa một mẫu cụ thể. Xác nhận DLL được nạp cần đối chiếu module list, thread và sự kiện runtime.

Nguồn: Elastic, Figure 1: “Rebhip worm performing a typical DLL injection” (Hosseini, 2017).

Mô hình tổng quát DLL injection



Lưu đồ phân tích DLL injection cổ điển



- Kiểm tra đường dẫn DLL, bytes được ghi và quyền của vùng nhớ.
- Nối thời điểm tạo thread với module load và hành vi sau đó.
- Biến thể có thể dùng cơ chế kích hoạt khác; lưu đồ này không đại diện cho mọi kiểu DLL injection.

Nguồn: Diễn giải khái niệm từ Elastic (Hosseini, 2017).

Bằng chứng cần thu thập khi thấy injection

- Quan hệ process: ai mở handle đến ai, với quyền nào và vào thời điểm nào.
- Memory map: vùng mới có quyền RWX hay chuyển từ RW sang RX, địa chỉ bắt đầu và nội dung.
- Module/thread: DLL nào xuất hiện, thread start address nằm trong module nào.
- Log hỗ trợ: Sysmon, ETW, debugger hoặc snapshot trước và sau.

Tình huống: DLL hợp lệ hay đáng ngờ?

Nhóm 3 người · 6 phút

Một trình đọc PDF nạp `C:\Program Files\Reader\codec.dll`. Sau đó một process khác ghi vào memory của trình đọc và tạo thread bắt đầu tại vùng không thuộc module. Hãy tách quan sát, suy luận và câu hỏi cần kiểm chứng.

Sản phẩm

Timeline 4 dòng, một kết luận tạm thời và hai giới hạn.

Phân tích tình huống

- Quan sát: DLL có đường dẫn hợp lệ; process khác ghi memory; thread bắt đầu ngoài module.
- Suy luận: dấu hiệu phù hợp với code injection, mạnh hơn chỉ báo “DLL lạ”.
- Cần kiểm chứng: nội dung vùng nhớ, nguồn process, quyền handle và kết quả thực thi.
- Giới hạn: chưa có hash/chữ ký DLL, chưa biết thread có chạy thành công và chưa có bản đối chứng.

Injection cần những quyền gì?

- Quyền truy cập process quyết định có thể đọc, ghi hoặc tạo thread hay không.
- Quyền bộ nhớ mô tả trạng thái reserve, commit, protection và transition.
- Cần ghi lại lỗi hoặc thành công của từng thao tác, không chỉ gọi tên API.

Lưu đồ DLL injection

- 1 Xác định process đích và mở handle.
- 2 Cấp phát vùng chứa đường dẫn hoặc payload.
- 3 Ghi dữ liệu và thay đổi protection nếu cần.
- 4 Kích hoạt loader hoặc thread.
- 5 Xác nhận module, thread và hành vi sau sự kiện.

DLL search order và rủi ro nạp nhầm

Nguyên tắc

Tên DLL không xác định file nào được nạp. Loader còn xét đường dẫn, thứ tự tìm kiếm, kiến trúc và dependency.

- So sánh đường dẫn thực tế với vị trí mong đợi.
- Kiểm tra DLL cùng tên trong thư mục ứng dụng.
- Kiểm tra chữ ký, hash và module phụ thuộc.

Thứ tự tìm DLL trong ứng dụng unpackaged

Giả định của lưu đồ

DLL được nạp theo tên mô-đun, ứng dụng desktop unpackaged, Safe DLL Search Mode bật và không có thay đổi search path. Redirection hoặc manifest có thể làm thay đổi kết quả.

- 1 DLL redirection, API sets, SxS manifest redirection.
- 2 Danh sách module đã nạp, rồi Known DLLs.
- 3 Package dependency graph trên Windows 11 21H2 trở lên.
- 4 Thư mục ứng dụng, System32, system directory 16-bit, Windows.
- 5 Thư mục hiện hành, rồi các thư mục trong PATH.

Lưu ý

Đường dẫn đầy đủ, LoadLibraryEx, SetDllDirectory và cờ LOAD_LIBRARY_SEARCH có thể đổi tập hoặc thứ tự tìm. DLL phụ thuộc lại được tìm theo tên mô-đun.

IAT, export và điểm quan sát

- Import table mô tả dependency tĩnh.
- IAT chứa địa chỉ hàm được resolve trong process.
- Export table cho biết API module cung cấp.
- Import là khả năng, IAT là kết quả resolve, trace là bằng chứng sử dụng.

4

Process hollowing

Câu hỏi trọng tâm

Một process hợp lệ có thể trở thành vỏ bọc cho image khác như thế nào?

Định nghĩa process hollowing

Định nghĩa

Process hollowing là kỹ thuật tạo một process hợp lệ ở trạng thái tạm dừng, tháo hoặc bỏ qua image ban đầu, ghi image khác vào không gian địa chỉ và chuyển luồng thực thi sang image mới.

- Vỏ process và image thực thi có thể không cùng nguồn.
- Parent-child relationship có thể trông bình thường nếu chỉ nhìn process tree.
- Cần đổi chiếu image trên đĩa, PEB, memory map và thread start address.

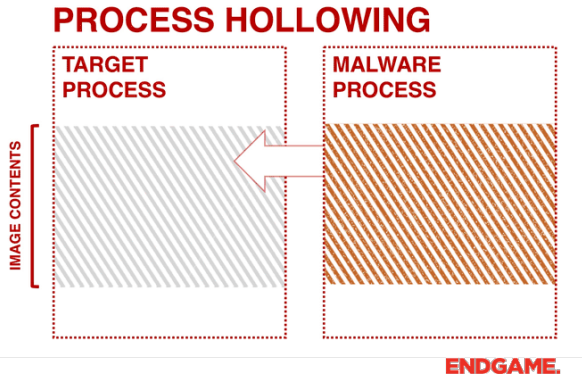
Chuỗi trạng thái cần tái dựng

- 1 Tạo process hợp lệ với cờ suspended.
- 2 Unmap hoặc thay vùng image ban đầu.
- 3 Cấp phát và ghi PE khác vào memory.
- 4 Sửa context, entry point hoặc PEB.
- 5 Resume thread và quan sát hành vi.

Câu hỏi

Ở bước nào có thể thu được bằng chứng động rõ nhất? Ở bước nào memory forensics giúp phân biệt image?

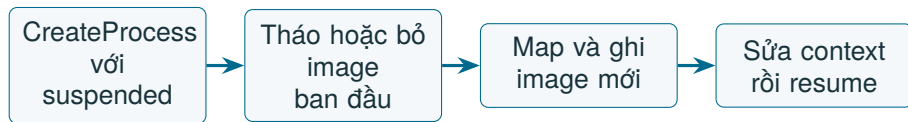
Process hollowing trong hình động



- Hình đối chiếu image của process đích với image được thay vào.
- Image ban đầu bị tháo hoặc bỏ, sau đó không gian đích chứa image mới.
- Đây là sơ đồ khái niệm; không phải mọi biến thể đều dùng cùng trình tự API.

Nguồn: Phỏng theo hình động “Process Hollowing”, Elastic, Hosseini (2017).

Lưu đồ: từ process suspended đến thực thi



- Trước resume, primary thread chưa chạy image mới.
- Theo dõi process creation, unmap/write, context và thời điểm resume.

Nguồn: Chuỗi khái niệm tổng hợp từ mô tả kỹ thuật của Elastic (2017).

So sánh DLL injection và hollowing

Tiêu chí	DLL injection	Process hollowing
Image ban đầu	Vẫn chạy, thường nạp thêm module	Bị thay hoặc bị che bởi image khác
Dấu vết chính	Module mới, vùng nhớ và thread	Sai khác giữa file image, PEB, VAD và entry point
Câu hỏi cốt lõi	Ai đưa mã vào process đang chạy?	Process đang chạy thật sự là image nào?

Kiểm tra nhanh: chọn bằng chứng mạnh hơn

Cặp đôi · 4 phút

Một process `svchost.exe` có command line bình thường nhưng memory chứa PE header không khớp với file trên đĩa. Bằng chứng này gợi ý điều gì và cần kiểm tra thêm gì?

Đáp án có giải thích

Kết luận tạm thời

Có sai khác giữa image trên đĩa và image trong memory, phù hợp với hollowing hoặc một cơ chế thay đổi memory khác.

- Kiểm tra PEB.ImageBaseAddress, VAD, các section và thread start address.
- So sánh hash vùng nhớ với file trên đĩa, ghi lại thời điểm dump.
- Loại trừ JIT, unpacking hoặc cơ chế cập nhật hợp lệ bằng mẫu đối chứng.

Nguyên tắc tái dựng hollowing

- Xác định process được tạo ở trạng thái nào.
- So sánh image path, PEB, VAD và module list.
- Ghi instruction pointer trước và sau resume.
- Đối chiếu vùng memory với file trên đĩa.

Lưu đồ kiểm chứng image-memory

- 1 Thu hash và PE metadata trên đĩa.
- 2 Chụp module map và PEB.
- 3 Dump vùng nghi vấn.
- 4 So sánh header, section, entry point.
- 5 Viết kết luận kèm giới hạn.

VAD và vùng private

VAD

Cấu trúc mô tả vùng địa chỉ ảo mà hệ điều hành quản lý cho process.

- Vùng private RX ngoài module là tín hiệu cần điều tra.
- Không tự động đồng nghĩa injection vì unpacking và JIT cũng có thể tạo vùng này.

Thread start address

- Xác định địa chỉ bắt đầu của thread.
- Ánh xạ địa chỉ đó vào module, section hoặc vùng private.
- Kiểm tra call stack và thời điểm tạo thread.
- Kết hợp với protection transition để tăng độ tin cậy.

Phân biệt hollowing với unpacking

Tiêu chí	Hollowing	Unpacking
Process creation	Thường có image vỏ	Stub của chính image
Nguồn payload	Image khác	Payload được giải nén
Bằng chứng	File–memory mismatch	Stub, vùng giải mã, luồng gọi

Bài tập: chọn dữ liệu ưu tiên

Cặp đôi · 5 phút

Chọn ba dữ liệu ưu tiên khi chỉ có 10 phút: process tree, PEB, VAD, dump memory, command line, hash file. Giải thích vì sao.

Đáp án ưu tiên dữ liệu

Một lựa chọn hợp lý

PEB, VAD và dump memory.

- Ba nguồn nối image runtime với vùng nhớ thực tế.
- Process tree và command line vẫn cần để dựng timeline.
- Hash file cần để đối chiếu nhưng không đủ trả lời image đang chạy là gì.

Context thread và thanh ghi

- Instruction pointer cho biết lệnh kế tiếp.
- Stack pointer và call stack giúp xác định đường gọi.
- Context thay đổi có thể là bước chuyển sang image/payload mới.
- Luôn lưu snapshot trước và sau sự kiện.

Dấu vết âm tính không phải bằng chứng phủ định

Ví dụ

Không thấy CreateRemoteThread không loại trừ injection dùng thread có sẵn, callback hoặc kỹ thuật khác.

- Ghi rõ phạm vi công cụ và thời điểm quan sát.
- Phân biệt “không quan sát thấy” với “không xảy ra”.

Tổng hợp phần process hollowing

- Hollowing thay đổi image mà process mang trong memory.
- PEB, VAD, context, thread và file trên đĩa tạo thành chuỗi bằng chứng.
- Kết luận phải loại trừ unpacking, JIT và thay đổi hợp lệ.

5

Đối chiếu bằng chứng

Câu hỏi trọng tâm

Khi nào một chuỗi quan sát đủ mạnh để gọi là injection?

Ba lớp bằng chứng

Tĩnh

PE, section, import, strings, chữ ký, hash.

Động

Process tree, API trace, file/registry, network, thread.

Memory

VAD, quyền trang, module list, PE header, thread start.

Nguyên tắc

Mỗi kết luận quan trọng nên có ít nhất hai nguồn độc lập hoặc nêu rõ vì sao chưa có nguồn thứ hai.

Timeline mẫu cho một ca phân tích

Thời điểm	Quan sát	Ý nghĩa có điều kiện
10:01:02	parent.exe mở handle đến target.exe	Có khả năng chuẩn bị thao tác trên đích.
10:01:03	Vùng memory mới chuyển RW → RX	Có thể chứa mã thực thi, cần dump.
10:01:03	Thread bắt đầu tại vùng ngoài module	Củng cố giả thuyết injection.

Sai lầm thường gặp

- Gọi mã độc chỉ vì thấy `CreateRemoteThread`.
- Nhầm DLL loading hợp lệ với DLL injection.
- Kết luận “fileless” vì không thấy file mới, dù vẫn có log hoặc script trên đĩa.
- Bỏ qua thời gian, quyền truy cập và process tạo ra hành vi.

Cách sửa

Viết theo ba cột: quan sát được, suy luận, điều kiện cần kiểm chứng.

Bài tập tổng hợp có hướng dẫn

Nhóm 3 người · 10 phút

Cho bốn mảnh bằng chứng: (A) import LoadLibrary; (B) process A mở handle vào B; (C) B có vùng RX ngoài module; (D) thread của B bắt đầu tại vùng đó. Xếp mức độ mạnh và dựng kết luận tối đa 3 câu.

Đáp án bài tập tổng hợp

- 1 A là chỉ báo yếu vì import chỉ cho biết khả năng.
- 2 B cho biết quan hệ tác động nhưng chưa chỉ ra dữ liệu nào được ghi.
- 3 C và D kết hợp là bằng chứng mạnh hơn cho mã ngoài module được thực thi trong B.

Kết luận mẫu

Chuỗi B–C–D phù hợp với injection vào B; cần dump vùng nhớ và kiểm tra nguồn process để xác nhận cơ chế cụ thể.

Từ API đến hành vi

- 1 API là sự kiện quan sát.
- 2 Xác định đối tượng, quyền và tham số.
- 3 Kiểm tra trạng thái memory sau sự kiện.
- 4 Tìm thread/module/hành vi nối tiếp.

Timeline và quan hệ nhân quả

- Timestamp cần cùng múi giờ và độ phân giải đủ.
- Sắp xếp sự kiện trước khi giải thích.
- Không gán nguyên nhân cho sự kiện xảy ra sau.

Đối chứng lành

- Chạy bản lành cùng phiên bản.
- Giữ nguyên môi trường và thao tác.
- So sánh khác biệt thay vì chỉ tìm dấu hiệu bất thường.

Bảng quan sát–suy luận

Quan sát	Suy luận	Cần kiểm chứng
Vùng RX ngoài module	Có code riêng trong memory	Dump và disassemble
Ghi memory vào process khác	Có tác động liên tiến trình	Quyền, bytes và kết quả
File hash khác baseline	Tệp đã thay đổi	Diff PE và nguồn thay đổi

Một kết luận tốt gồm bốn phần

- 1 Mệnh đề chính.
- 2 Bằng chứng trực tiếp.
- 3 Điều kiện hoặc giả định.
- 4 Giới hạn và bước tiếp theo.

Tình huống phản biện kết luận

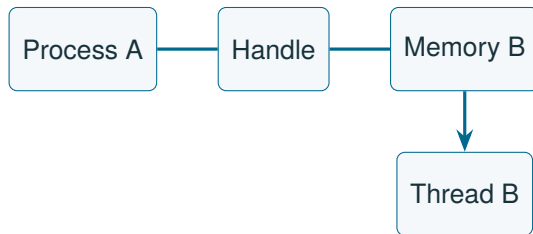
Nhóm 3 người · 6 phút

“Có WriteProcessMemory nên chắc chắn payload đã chạy.” Hãy chỉ ra phần đúng, phần vượt quá bằng chứng và dữ liệu cần bổ sung.

Đáp án phản biện

- Đúng: đã có thao tác ghi vào không gian process đích.
- Vượt quá: chưa biết bytes là gì và có được thực thi không.
- Bổ sung: protection, thread/callback, dump memory, call stack và hành vi sau đó.

Sơ đồ evidence graph



Ý nghĩa

Mỗi cạnh phải có nguồn: trace, log hoặc snapshot.

6

Tình huống thực hành và exit ticket

Câu hỏi trọng tâm

Từ dấu hiệu rời rạc đến một kết luận có thể kiểm chứng

Quy trình phân tích trong lab

- 1 Tạo snapshot và ghi hash mẫu ban đầu.
- 2 Quan sát PE và lập giả thuyết về điểm chèn hoặc điểm kích hoạt.
- 3 Chạy mẫu trong môi trường cô lập, ghi process/module/thread.
- 4 Chụp memory map trước và sau sự kiện đáng chú ý.
- 5 Đối chiếu file, memory và timeline, sau đó viết giới hạn.

An toàn

Chỉ dùng mẫu minh họa do giảng viên cung cấp; không kết nối mạng thật và không tác động process ngoài phạm vi lab.

Phiếu phân tích một trang

Câu hỏi	Trả lời bằng bằng chứng cụ thể
Mẫu đã thay đổi ở đâu?	File, section, entry point, memory hay module?
Sự kiện nào xảy ra trước?	Timeline với thời gian và nguồn log.
Cơ chế phù hợp nhất?	Virus file, DLL injection, hollowing hoặc chưa đủ dữ liệu.
Điều chưa biết?	Nêu dữ liệu cần thu thêm và ảnh hưởng đến kết luận.

Exit ticket

Cá nhân · 5 phút, nộp cuối buổi

Trả lời ngắn nhưng có lý do:

1. Phân biệt file infection và process injection bằng một câu cho mỗi khái niệm.
2. Chọn hai bằng chứng mạnh nhất để kiểm tra injection và giải thích vì sao.
3. Một quan sát nào dễ dẫn đến kết luận sai? Viết cách kiểm chứng.

Đáp án tham khảo exit ticket

- File infection sửa tệp để lần chạy sau chuyển điều khiển; injection đưa mã hoặc dữ liệu vào process đang chạy.
- Vùng memory thực thi ngoài module kết hợp với thread start address và timeline process là bằng chứng mạnh hơn một API đơn lẻ.
- “Có DLL lạ” chỉ là chỉ báo; cần đường dẫn, hash, chữ ký, nguồn nạp và hành vi sau khi nạp.

Tóm tắt Buổi 3

- Virus file tác động lên tệp và luồng thực thi của tệp.
- DLL injection và hollowing tác động lên không gian địa chỉ hoặc image của process.
- Kết luận đáng tin cần nối file, process, memory và thời gian.
- Mọi suy luận phải nêu bằng chứng, điều kiện và giới hạn.

Cầu nối

Buổi 4 mở rộng từ một process bị tác động sang cơ chế worm lan truyền qua nhiều nút mạng.

Tài liệu tham khảo I



Abhijit Mohanta, Anoop Saldanha.
emphMalware Analysis and Detection Engineering. Apress, 2020.



Michael Sikorski, Andrew Honig.
emphPractical Malware Analysis. No Starch Press, 2012.



Mark Russinovich et al.
emphWindows Internals, Part 1. Microsoft Press, 2021.



Chris Eagle, Kara Nance.
emphThe Ghidra Book. No Starch Press, 2020.



Sami Laiho et al.
emphWindows Internals and Memory Forensics. Tài liệu đọc bổ trợ trong học phần.